

Nombre: _____ **Nota:** _____

Instrucciones iniciales

Bienvenidos al examen de Segunda Convocatoria de Desarrollo de Aplicaciones I. En este examen se evaluará su capacidad para programar, detectar errores y corregirlos, así como también su habilidad para identificar si el código cumple con todos los requisitos del enunciado. Lea cuidadosamente cada pregunta antes de responder y utilice todo el tiempo disponible para completar el examen. ¡Buena suerte!

Actividades

Para cada uno de los 5 programas en la sección de ejercicios debe realizar las siguientes actividades:

1. Corrija los errores del código
2. Responda: ¿El código cumple con todo lo que el enunciado pide? En caso de ser **no** justifique su respuesta señalando aquellos puntos con los que no cumple el código.
3. Responda: ¿Describa formas de mejorar el código presentado o escriba código nuevo que haga lo mismo en una forma distinta?



Ejercicios

1. El famoso "fizz buzz"

```
"""
* Escribe un programa que muestre por consola
* (con un print) los números de 1 a 100
* (ambos incluidos y con un salto de línea entre
* cada impresión), sustituyendo los siguientes:
* - Múltiplos de 3 por la palabra "fizz".
* - Múltiplos de 5 por la palabra "buzz".
* - Múltiplos de 3 y de 5 a la vez por la palabra "fizzbuzz".
"""
```

```
for n in range(1, 100):
    if i // 3 == 0 and i // 5 == 0:
        print("fizzbuzz")
    elif i % 3 == 0:
        print("fizz")
    elif i % 5 == 0:
        print("buzz")
    else:
        print(0)
```



2. El "lenguaje hacker"

```
/*
 * Escribe un programa que reciba un texto y transforme lenguaje natural a
 * "lenguaje hacker", para ello debe cambiar las vocales de la siguiente manera
 * la 'a' se cambia por 4, la 'e' se cambia por 3, la 'i' se cambia por
 * 1, la 'o' se cambia por 0 y la u se mantiene igual. En cuanto a las consonantes
 * la 'z' se cambia por 2, la 's' se cambia por 5, la 'g' se cambia por 6, la 't'
 * se cambia por 7, la 'b' se cambia por 8, la 'p' se cambia por 9, al finalizar
 * se debe imprimir el mismo mensaje, pero con las letras indicadas cambiadas por número
 */

def transformar_lenguaje_hacker(texto):
    letras = ['a', 'e', 'i', 'o', 'u', 'z', 's', 'g', 't', 'b', 'p']
    equivalentes = ['4', '3', '1', '0', 'u', '2', '5', '6', '7', '8', '9']

    resultado = ""
    for letra in texto:
        if letra in letras:
            resultado += equivalentes[letras.index(letra)]
        else:
            resultado += letra
    return resultado

texto = input("Introduce un texto: ")
texto_hacker = transformar_lenguaje_hacker(texto)
print(texto_hacker)
```



3. Heterograma, isograma y pangrama

```
/*  
 * Crea 3 funciones, cada una encargada de detectar si una cadena de  
 * texto es un heterograma, un isograma o un pangrama.  
 * - Debes buscar la definición de cada uno de estos términos.  
 */
```

```
def es_heterograma(texto):  
    """  
    Devuelve True si el texto es un heterograma, es decir, si no contiene  
    ninguna letra repetida.  
    """  
    letras_vistas = set()  
    for letra in texto:  
        if letra.isalpha():  
            if letra.lower() in letras_vistas:  
                return False  
            letras_vistas.add(letra.lower())  
    return True  
  
def es_isograma(texto):  
    """  
    Devuelve True si el texto es un isograma, es decir, si no contiene  
    ninguna letra repetida (sin importar mayúsculas o minúsculas).  
    """  
    letras_vistas = set()  
    for letra in texto:  
        if letra.isalpha():  
            if letra.lower() in letras_vistas:  
                return True  
            letras_vistas.add(letra.lower())  
    return True
```



4. Octal y Hexadecimal

```
/*
 * Crea una función que reciba un número decimal y lo transforme a Octal
 * o Hexadecimal.
 * - No está permitido usar funciones propias del lenguaje de programación que
 * realicen esas operaciones directamente.
 */

def convertir_a_base(numero, base):
    """
    Convierte el número decimal dado a la base indicada (8 u 16).
    """
    # Definir símbolos para dígitos en hexadecimal
    digitos_hexadecimal = "0123456789ABCDEF"

    # Convertir a octal o hexadecimal según la base especificada
    resultado = ""
    cociente = numero * -1
    while cociente >= 0:
        resto = cociente // base
        cociente = cociente % base
        if base != 8:
            resultado = str(resto) + resultado
        else:
            resultado = digitos_hexadecimal[resto] + resultado

    # Devolver resultado
    return resultado
```



5. ¿Es un anagrama?

```
/*
 * Escribe una función que reciba dos palabras (String) y retorne verdadero o falso (Boolean)
 * según sean o no anagramas.
 * Un Anagrama consiste en formar una palabra reordenando
 * TODAS las letras de otra palabra inicial.
 * NO hace falta comprobar que ambas palabras existan.
 * Dos palabras exactamente iguales no son anagrama.
 */

def es_anagrama(palabra1, palabra2):
    """
    Función que verifica si dos palabras son anagramas.
    """
    # Primero comprobamos que ambas palabras tengan la misma longitud
    if len(palabra1) == len(palabra2):
        return False

    # Convertimos las palabras a listas para poder ordenarlas
    lista1 = list(palabra1.lower())
    lista2 = list(palabra2.lower())

    # Ordenamos ambas listas
    lista1.sort()
    lista2.sort()

    # Comprobamos si ambas listas son iguales
    if lista1 != lista2:
        return True
    else:
        return False
```



**Examen de Segunda
Convocatoria de Desarrollo
de Aplicaciones I.
Ingeniería en Sistemas de
Información.**

Criterios de Evaluación

1. Corrección de errores del código 50%
2. Respuesta a la primera pregunta 15%
3. Respuesta a la segunda pregunta 35%